Sprint 2 – Week 1 Progress Report

SWENG 894 Capstone

MicroMatch

GitHub Repository Link

Chastidy Joanem

1. System Overview and Context

1.1 System Scope

MicroMatch is a web-based micro-internship marketplace designed to connect students with short-term, skill-based professional opportunities. The platform enables:

- Organizations to post structured, well-defined micro-projects.
- Students to browse and discover opportunities.
- Students to apply for projects.
- Organizations to manage and review applications.
- A controlled workflow to manage project participation.

The system architecture follows a layered full-stack model:

- **Frontend:** React-based SPA (Single Page Application)
- **Backend:** FastAPI REST API
- **Database:** PostgreSQL (relational persistence)
- **Authentication:** JWT-based stateless authentication
- **Authorization:** Role-Based Access Control (RBAC)

The platform supports two primary user roles:

- Student
- Organization

1.2 Sprint 1 Recap – MVP Foundation

Sprint 1 delivered the Minimum Viable Marketplace (MVP Foundation), including:

- User authentication and registration (JWT authentication)
- Role-Based Access Control (Student / Organization)
- Student profile management
- Organization profile management
- Project posting
- Basic project browsing and search

User Story	Description
US-1	Registration/Login
US-2	Student Profile

US-3	Organization Profile
US-4	Post Listing
US-5	Browse/Search

These stories satisfied:

- FR-1 through FR-6
- End-to-end workflow from account creation to project discovery

Sprint 1 delivered authentication, profile management, posting, and browsing functionality. Sprint 2 introduces the application workflow layer, enabling real interaction between students and organizations.

1.3 Functional Requirements to User Stories Mapping

Functional Requirement	User Story
FR-1: The system shall allow users to create an account.	US-1
FR-2: The system shall support role-based access control.	US-1
FR-3: The system shall allow students to create/edit profile.	US-2
FR-4: The system shall allow organizations to create profile.	US-3
FR-5: The system shall allow orgs to post listings.	US-4
FR-6: The system shall allow students to browse/search listings.	US-5
FR-7: The system shall allow students to apply.	US-6
FR-8: The system shall track application status.	US-8
FR-9: The system shall allow orgs to accept/reject.	US-9
FR-10: The system shall allow students to submit deliverables.	US-10
FR-11: The system shall allow orgs to review deliverables.	US-11

FR-12: The system shall allow marking projects complete.	US-12
FR-13: The system shall allow feedback submission.	US-13
FR-14: The system shall allow admin moderation.	US-14
FR-15: The system shall log system actions.	US-15

This traceability ensures alignment between formal IEEE-style functional requirements and Agile user stories used for sprint planning.

2. Sprint 2 – Week 1 Overview

2.1 Sprint Goal

The goal of Sprint 2 – Week 1 is:

- To implement the core Application Workflow, enabling students to apply to projects and view their applications through a structured backend and frontend integration.

This sprint transitions the platform from a "discover-only" marketplace to an interactive workflow system.

3. Sprint Planning

3.1 Product Backlog Grooming and Reprioritization

At the beginning of Sprint 2, I conducted backlog refinement to prioritize the next critical features following Sprint 1's MVP delivery. The system was capable of discovery, but lacked interaction. Therefore, the next logical milestone was implementing the Application Workflow.

Reprioritization Strategy

I used Planning Poker with Fibonacci estimation (3, 5, 8, 13) to reassess story complexity and confirm story point allocations.

The highest priority story identified was:

- US-6: Apply (5 points)

Rationale:

Without application submission, the marketplace has no transactional value.

US-8 (Track Status) was partially selected because status visibility is essential immediately after submission.

3.2 Groomed Product Backlog

Priority Definitions

- High: Core MVP functionality required for end-to-end workflow.
- Medium: Important workflow enhancements dependent on core features.
- Low: Administrative or cross-cutting features not required for initial MVP validation.

ID	User Story	Priority	Story Points	Sprint	Rationale
US-1	Registration/Login	High	5	Sprint 1	Required for all access
US-2	Student Profile	High	5	Sprint 1	Core student data
US-3	Org Profile	High	5	Sprint 1	Required before posting
US-4	Post Listing	High	8	Sprint 1	Core marketplace feature
US-5	Browse/Search	High	5	Sprint 1	Enables discovery
US-6	Apply	High	5	Sprint 2	Core interaction
US-7	Review Applications	Medium	5	Sprint 2	Depends on applications
US-8	Track Status	High	3	Sprint 2	Enhances UX
US-9	Accept/Reject	Medium	3	Sprint 2	Workflow control
US-10	Submit Deliverables	Medium	5	Sprint 3	Completion flow
US-11	Review Deliverables	Medium	3	Sprint 3	Completion validation

US-12	Mark Complete	Medium	3	Sprint 3	Project closure
US-13	Feedback	Low	5	Sprint 3	Post-completion feature
US-14	Admin Moderation	Low	5	Sprint 3	Governance feature
US-15	Logging	Low	8	Sprint 3	Cross-cutting concern

3.3 Sprint Backlog – Week 1 Commitment

Based on my capacity, Week 1 included:

User Story	Description	Story Points
US-6	Apply to Project	5
US-8	Track Status (Student View)	3
Supporting Tasks	Model + UI Integration	9

Total Commitment: 17 Story Points

The sprint commitment was intentionally limited to the student-side workflow to avoid introducing parallel complexity (organization review logic). By isolating application submission and status retrieval, the sprint minimized cross-feature dependencies and reduced integration risk. This phased rollout aligns with Agile incremental delivery principles.

Capacity Planning and Velocity Justification

As a single-developer team, historical velocity from Sprint 1 averaged approximately 15–18 story points per week, based on completed authentication, profile, and listing features.

Given:

- Backend + frontend integration work
- New database model
- New API endpoints
- UI dashboard implementation
- Unit test coverage requirement (≥95%)

A commitment of 17 story points was determined to be:

- Ambitious but achievable
- Within historical velocity range

- Balanced between feature development and quality assurance effort

US-7 and US-9 were deferred to Week 2 due to:

- Dependency on stable application model
- Need to validate submission workflow before enabling decisions

This reflects risk-controlled incremental development.

3.4 Sprint 2 Backlog Structure

Sprint 2 was divided into three weekly development phases.

Week 1 – Application Creation (Student Side)

Primary Focus:

- Allow students to apply to projects
- Provide application visibility
- Prevent duplicate submissions
- Integrate frontend UI workflow

Completed Issues

Issue	Description	Story Points	Status
#45	Create Application Database Model	3	Done
#46	Implement Apply to Project Endpoint	5	Done
#47	Implement Get My Applications Endpoint	3	Done
#49	Frontend Apply Button Integration	3	Done
#50	Student Applications Dashboard	3	Done

Week 1 Total Commitment: 17 Story Points

Week 2 – Organization Review Workflow

Primary Focus:

- Application visibility for organizations
- Status decision workflow

Planned Issues

Issue	Description	Story Points
#48	Organization View Applications Endpoint	5
#53	Accept / Reject Endpoint	5
#54	Organization Dashboard Integration	3

Week 2 Total Commitment: 13 Story Points

Week 3 — Workflow Completion & Stability

Primary Focus:

- Status synchronization
- Business rule enforcement
- Integration testing

Planned Issues

Issue	Description	Story Points
#56	Status Badge Sync	3
#55	Prevent Multiple Accepted Applications	3
#57	End-to-End Workflow Testing	5

Week 3 Total Commitment: 11 Story Points

4. Definition of Done (Sprint 2)

The Definition of Done was refined for Sprint 2 and applied consistently to all backlog items.

A User Story is considered complete when:

Design

- Data model designed and reviewed.
- API contract documented.

Implementation

- Database migration applied.
- Backend endpoint implemented.
- Role-based access control enforced.

- Validation rules implemented.
- Frontend integration completed (if applicable).

Testing

- Unit tests written.
- Edge case tests implemented.
- Manual functional validation completed.
- No failing tests.

Quality & Integration

- Coverage ≥ 95%.
- Code reviewed.
- PR merged into main branch.
- No regression introduced to Sprint 1 features.

5. Acceptance Criteria

5.1 US-6 – Apply

Acceptance Criteria:

- Only authenticated students may apply.
- Project must exist.
- Project must be open.
- Duplicate applications prevented.
- Status defaults to “pending”.
- Returns:
    - 201 on success
    - 400 if duplicate
    - 403 if wrong role
    - 404 if project not found

5.2 US-8 – Track Status (Student Side)

Acceptance Criteria:

- Student-only access.
- Endpoint returns:
    - Project title
    - Organization name
    - Status
    - Application date
- Returns empty list if no applications exist.
- Dashboard renders:
    - Status badge
    - Timestamp
    - Project reference

6. Source Code Development Progress

6.1 Repository

GitHub Repository: https://github.com/chazdj/MicroMatch

Project board integrated with GitHub Issues for traceability between user stories and commits.

6.2 Summary of Contributions

Backend Contributions

- Designed and implemented Application ORM model.
- Enforced unique constraint (student_id + project_id).
- Implemented POST /projects/{project_id}/apply.
- Implemented GET /applications/me.
- Added duplicate validation logic.
- Enforced RBAC using dependency injection.

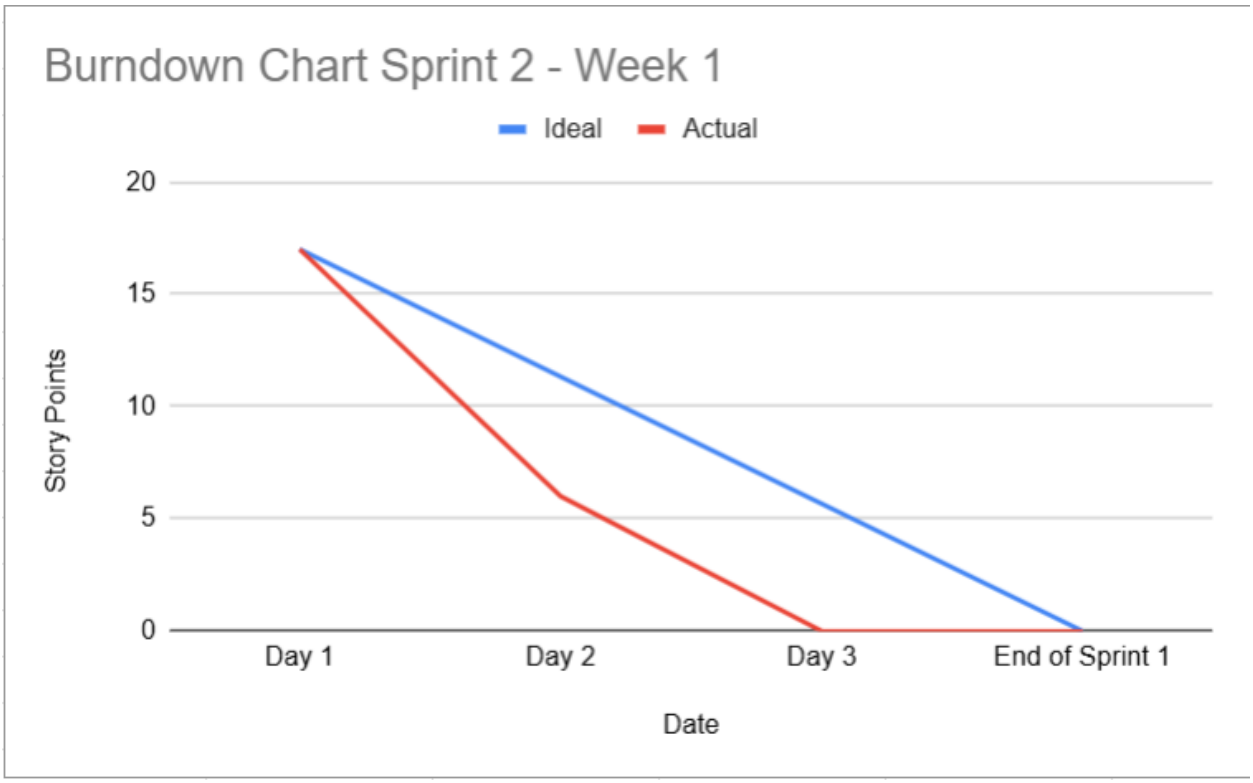
Frontend Contributions

- Integrated Apply button with backend endpoint.
- Implemented conditional rendering (student-only visibility).
- Created Student Applications Dashboard.
- Implemented loading states and error handling.

6.3 Important Commits

Commit Message	Related Issue	User Story
feat: create application database model	#45	US-6
feat: apply endpoint implementation	#46	US-6
feat: applications retrieval endpoint	#47	US-8
ui: integrate apply button	#49	US-6
ui: student dashboard implementation	#50	US-8

6.4 Burndown Chart



Sprint Capacity: 17 Story Points
Sprint Duration: 3 Development Days

Burndown Chart Analysis

The burndown chart demonstrates steady progress across the sprint.

- Early reduction reflects completion of backend model and apply endpoint.
- Mid-sprint decline corresponds to frontend integration.
- Final drop to zero reflects dashboard completion and test validation.

There were no scope increases or mid-sprint additions, resulting in a clean burn trajectory.

The sprint completed all committed story points within the planned timeline, indicating accurate estimation and sustainable workload.

Velocity Alignment:

Sprint 1 average velocity: ~16 points
Sprint 2 Week 1 completed: 17 points

7. Software Testing

7.1 Requirement-to-Test Mapping

Functional Requirement	User Story	Test Case ID
FR-7	US-6	TC-US6-01
FR-7	US-6	TC-US6-02
FR-7	US-6	TC-US6-03
FR-7	US-6	TC-US6-04
FR-7	US-6	TC-US6-05
FR-8	US-8	TC-US8-01
FR-8	US-8	TC-US8-02

7.2 User Acceptance Test Specification

All test cases are mapped to FR-7 and FR-8 ensuring full coverage of sprint scope.

TC-US6-01 – Successful Application Submission

Related Requirement: FR-7
User Story: US-6 – Apply
Actor: Student

Preconditions:

- Student account exists and is authenticated.
- Project exists in the database.
- Project status = "open".
- Student has not previously applied to the project.

Steps:

1. Log in as a Student.
2. Navigate to the Project Details page.
3. Verify the “Apply” button is visible.
4. Click the “Apply” button.
5. Confirm submission in the confirmation modal (if applicable).
6. System sends POST `/projects/{project_id}/apply` request.

Expected Result:

- HTTP 201 Created is returned.
- A new application record is created in the database.
- student_id and project_id are correctly stored.
- status field defaults to "pending".
- The Apply button becomes disabled or hidden.
- The application appears in the Student Applications Dashboard.
- The dashboard displays:
    - Project title
    - Organization name
    - Status = "pending"
    - Correct timestamp

Postconditions:

- Application persists in database.
- No duplicate entries exist.

TC-US6-02 – Duplicate Application Prevention

Related Requirement: FR-7

User Story: US-6 – Apply

Actor: Student

Preconditions:

- Student is authenticated.
- Project exists and is open.
- Student has already successfully applied to this project.

Steps:

1. Log in as the same Student.
2. Navigate to the same Project Details page.
3. Click the "Apply" button again.
4. System sends POST `/projects/{project_id}/apply`.

Expected Result:

- HTTP 400 Bad Request is returned.
- Error message displayed: "You have already applied to this project."
- No additional application record is created.
- Database contains exactly one application record for this student/project pair.
- Dashboard shows only one application entry.

Postconditions:

- System state remains unchanged.
- No duplicate records created.

TC-US6-03 – Unauthorized Role Attempt

Related Requirement: FR-7
User Story: US-6 – Apply
Actor: Organization User

Preconditions:

- Organization user account exists and is authenticated.
- Project exists and is open.

Steps:

1. Log in as an Organization.
2. Navigate to Project Details page.
3. Attempt to trigger application submission via:
    a. UI (if button visible), or
    b. Direct API request to POST `/projects/{project_id}/apply`.

Expected Result:

- HTTP 403 Forbidden is returned.
- Error message displayed: "Only students may apply."

- No application record created.
- System logs the unauthorized attempt (if logging enabled).

Postconditions:

- Database remains unchanged.
- No application stored.

TC-US6-04 – Project Not Found

Related Requirement: FR-7
User Story: US-6 – Apply
Actor: Student

Preconditions:

- Student authenticated.
- Project ID does not exist in database.

Steps:

1. Send POST request to `/projects/{invalid_project_id}/apply`.

Expected Result:

- HTTP 404 Not Found returned.
- Error message: "Project not found."
- No database entry created.

Postconditions:

- No data persisted.

TC-US6-05 – Apply to Closed Project

Related Requirement: FR-7
User Story: US-6 – Apply
Actor: Student

Preconditions:

- Student authenticated

- Project exists
- Project status = “closed”

Steps:

1. Log in as Student.
2. Navigate to closed project.
3. Click Apply.
4. System sends POST request.

Expected Results:

- HTTP 400 Bad Request returned.
- Error message: “Project is no longer accepting applications.”
- No application record created.
- Database unchanged.

Postconditions:

- No new application exists.

TC-US8-01 – Dashboard Displays Applications

Related Requirement: FR-8
User Story: US-8 – Track Status
Actor: Student

Preconditions:

- Student authenticated.
- Student has one or more applications stored.
- Applications contain various statuses (e.g., pending).

Steps:

1. Log in as Student.
2. Navigate to `/my-applications` dashboard.
3. System sends GET `/applications/me`.

Expected Results:

- HTTP 200 OK returned.

- Response payload contains:
    - Project title
    - Organization name
    - Status
    - Application date
- Dashboard renders a list of applications.
- Status is displayed as a badge.
- Data matches database records exactly.

Postconditions:

- No database changes occur.

TC-US8-02 – Empty State

Related Requirement: FR-8
User Story: US-8 – Track Status
Actor: Student

Preconditions:

- Student authenticated.
- Student has no applications in database

Steps:

1. Log in as Student.
2. Navigate to `/my-applications`.
3. System sends GET `/applications/me`.

Expected Result:

- HTTP 200 OK returned.
- Response payload = empty list.
- Dashboard displays:
- Message: "You have not applied to any projects yet."
- No UI errors occur.

Postconditions:

- System state unchanged.

8. Backlog Grooming

During Week 1:

- No new user stories were added.
- No story point changes were required.
- No scope expansion occurred.
- Sprint 2 structure remains intact.

US-7 and US-9 remain scheduled for Week 2.

The backlog remains stable and aligned with sprint objectives.

9. Risk Management

Risk	Mitigation
Duplicate submissions	DB-level unique constraint
Role misuse	RBAC enforcement
Inconsistent state	Centralized validation
Frontend/backend mismatch	Early integration

10. Sprint 2 Week 1 Outcome

Sprint 2 Week 1 successfully delivered the foundation of the application lifecycle.

- The system now supports:
- Application submission
- Duplicate prevention
- Student-side status visibility
- Full backend/frontend integration
- High test coverage
- Stable sprint execution

The sprint goal was fully achieved.